

SEMICONDUCTOR TRADE ANALYSIS

Imports Data — MySQL Pipeline & Loading Process | 2025 Documentation

Overview

This document describes the steps taken to move the cleaned U.S. semiconductor imports dataset (Cleaned Imports USA.csv) into a MySQL relational database. By designing a precise SQL schema, each column is assigned the correct data type, boolean values from the CSV are preserved as text, and trade values and quantities are stored in numeric formats suitable for aggregation. The imports_data table holds 373 rows of partner-level monthly shipment records and serves as the foundation for all subsequent analytical queries.

Step 1 — Creating the Database

Objective

Create a dedicated schema to organize all semiconductor trade data and keep it separate from other projects.

SQL Code

```
CREATE DATABASE semiconductors_db;
```

Key Design Choices

- A dedicated schema named semiconductors_db isolates semiconductor trade data from unrelated projects and ensures clean namespace management.
- All subsequent table creation and queries are executed inside this schema using USE semiconductors_db.

Step 2 — Defining the Table Schema

Objective

Define a precise SQL schema for the imports_data table, ensuring each column has the correct data type, boolean flags are stored as VARCHAR(5), and trade values use DECIMAL precision.

SQL Code

```
CREATE TABLE imports_data (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  typeCode CHAR(1) NOT NULL,  
  freqCode CHAR(1) NOT NULL,  
  refPeriodId CHAR(8) NOT NULL,  
  refYear SMALLINT NOT NULL,  
  refMonth TINYINT NULL,  
  period CHAR(6) NOT NULL,  
  reporterCode INT NOT NULL,  
  reporterISO CHAR(3) NOT NULL,  
  reporterDesc VARCHAR(150) NOT NULL,  
  flowCode CHAR(1) NOT NULL,  
  flowDesc VARCHAR(20) NOT NULL,  
  partnerCode INT NOT NULL,  
  partnerISO CHAR(3) NOT NULL,  
  partnerDesc VARCHAR(150) NOT NULL,  
  classificationCode VARCHAR(10) NOT NULL,  
  classificationSearchCode VARCHAR(10) NULL,  
  isOriginalClassification VARCHAR(5) NOT NULL DEFAULT 'True',  
  cmdCode VARCHAR(20) NOT NULL,  
  cmdDesc TEXT NOT NULL,
```

```

aggrLevel          SMALLINT NOT NULL,
isLeaf             VARCHAR(5) NOT NULL DEFAULT 'False',
customsCode        VARCHAR(20) NULL,
customsDesc        VARCHAR(300) NULL,
motCode            VARCHAR(10) NULL,
motDesc            VARCHAR(150) NULL,
qtyUnitCode        SMALLINT NOT NULL,
qtyUnitAbbr        VARCHAR(20) NOT NULL,
qty                DECIMAL(20,4) NULL,
isQtyEstimated     VARCHAR(5) NOT NULL DEFAULT 'False',
altQtyUnitCode     SMALLINT NULL,
altQtyUnitAbbr     VARCHAR(20) NULL,
altQty             DECIMAL(20,4) NULL,
isAltQtyEstimated  VARCHAR(5) NOT NULL DEFAULT 'False',
netWgt             DECIMAL(20,4) NULL,
isNetWgtEstimated  VARCHAR(5) NOT NULL DEFAULT 'False',
isGrossWgtEstimated VARCHAR(5) NOT NULL DEFAULT 'False',
cifvalue           DECIMAL(20,4) NULL,
fobvalue           DECIMAL(20,4) NULL,
legacyEstimationFlag VARCHAR(10) NULL,
isReported         VARCHAR(5) NOT NULL DEFAULT 'True',
isAggregate        VARCHAR(5) NOT NULL DEFAULT 'False',
qty_standard       DECIMAL(20,4) NULL,
qty_kg             DECIMAL(20,4) NULL,
data_quality       VARCHAR(20) NOT NULL
);

```

Key Design Choices

- **Boolean fields as text:** All flags (isOriginalClassification, isLeaf, isQtyEstimated, etc.) are defined as VARCHAR(5) with defaults 'True' or 'False'. This matches the CSV values directly and avoids conversion errors.
- **Numeric precision:** Trade values (cifvalue, fobvalue) and quantities (qty, netWgt, qty_standard, qty_kg) are stored as DECIMAL(20,4) to ensure accuracy for large monetary values and fractional weights.
- **Identifiers:** Country codes (reporterCode, partnerCode) are set as INT to handle large numeric IDs. ISO codes are stored as CHAR(3) for consistency.
- **Text fields:** Commodity descriptions (cmdDesc) are stored as TEXT to accommodate long strings, while shorter descriptors use VARCHAR.

Step 3 — Importing the Data

Objective

Load the cleaned CSV into the imports_data table using MySQL Workbench's import wizard and verify row counts.

SQL Code

```

-- Verification query after import
SELECT COUNT(*) FROM imports_data;

-- Spot-check alignment
SELECT * FROM imports_data LIMIT 10;

```

Result

Metric	Value
Rows loaded	373
Import method	MySQL Workbench Import Wizard

Metric	Value
Type errors	None

Key Design Choices

- With the schema precisely aligned to the CSV column structure, the import completed without type mismatches or truncation errors.
- Spot checks using `SELECT * FROM imports_data LIMIT 10` confirmed correct alignment of values across all columns.

Key Decisions and Rationale

- **Boolean simplification:** Storing flags as text avoids preprocessing and keeps the dataset consistent with the original CSV source.
- **Precision in trade values:** Using DECIMAL ensures that financial and weight calculations remain accurate at scale.
- **Schema transparency:** Each column name and type mirrors the CSV, making the table auditable and easy to cross-reference.
- **Future scalability:** The schema allows additional datasets (e.g., exports, other partner countries) to be imported into the same database for comparative analysis.

Data Source

[UN Comtrade](#) — Cleaned Imports USA.csv